

國立臺北科技大學 智慧自動化工程科
114 學年度
期末專題報告書

麥克納姆輪自主追蹤機器人

指導教授:莊政達 助理教授
班 級:日間部(五專) 智動二
作 者:陳兆臨

中華民國 115 年 6 月 25 日

目錄

一、前言	2
二、預期結果	3
三、麥克納姆輪運動學	3
四、系統架構與硬體設計	5
五、電腦視覺追蹤管線	8
六、軟體模組說明	10
七、執行結果與分析	12
八、程式碼與參數設定	14
九、專題貢獻說明	16
十、結論與心得	16

一、前言

隨著電腦視覺與嵌入式系統技術的快速演進，具備全向移動能力的自主機器人已逐漸成為智慧製造、倉儲物流與競賽機器人等領域的核心平台。麥克納姆輪 (Mecanum Wheel) 因其獨特的 45° 滾輪排列方式，使車體無需轉向即可同時實現前後、橫移與自轉等三自由度運動，具有極高的場地適應性。

本報告旨在利用 Raspberry Pi 5 與 Arduino Mega 2560 建構一套低成本、模組化的自主視覺追蹤機器人系統。系統模型依據專題需求，整合了以下三大核心子系統：

1. **電腦視覺模組 (Computer Vision Module)**：採用 OpenCV 進行 HSV 色彩空間分割，偵測紅色目標物並計算其於畫面中的相對位置。
2. **麥克納姆輪運動學 (Mecanum Kinematics)**：根據目標偏移量計算所需之縱向速度 V_y 、橫向速度 V_x 及旋轉角速度 W_z ，並透過反運動學混合公式換算為四輪各自的轉速指令。
3. **序列通訊介面 (Serial Communication Interface)**：Raspberry Pi 端將計算完成的輪速指令以結構化格式打包後，經 USB 序列埠傳送至 Arduino，由 Arduino 驅動各輪之 PWM 馬達控制器。

為驗證系統之穩健性，本專題設計了兩種運作模式：

- 情境一：手動遙控模式 (Manual Teleoperation) —— 透過 USB 遊戲手把直接控制四輪速度向量，驗證全向移動能力。

- 情境二：自主追蹤模式 (Autonomous Tracking Mode) ——系統偵測到紅色目標後自動計算追蹤指令，目標消失時自動停止，驗證視覺回授迴路的穩定性。

透過觀察機器人在不同場景下的追蹤行為，本研究將深入分析視覺參數與控制策略對系統響應的影響，並總結實驗之發現。

二、預期結果

本專題的核心目標是驗證所設計的視覺追蹤控制器，是否能驅動麥克納姆輪機器人精準地鎖定並趨近紅色目標物，同時維持全向移動的流暢性。根據系統設計，我預期達成以下成果：

1. **目標鎖定穩定性：**無論目標出現於畫面左側、右側或中央，系統應能在短暫的調整期後，使機器人車頭對準目標方向，並計算出合理的前進與轉向速度。
2. **誤差最小化：**隨著追蹤過程進行，目標在畫面中的橫向偏移量 (e_x) 應呈現逐漸收斂的趨勢，最終穩定於畫面中心線附近的極小範圍內，達成動態穩定狀態。
3. **控制穩定性：**輸出至四輪的 PWM 訊號應在目標鎖定後趨於穩定，不產生劇烈的震盪或不規則運動。當目標消失時，機器人能立即停止，確保安全性。
4. **全向移動驗證：**在手動遙控模式下，操作者應能透過搖桿直覺地控制機器人進行前後、橫移及原地旋轉，充分展現麥克納姆輪的全向移動優勢。

三、麥克納姆輪運動學

為了讓機器人能夠實現全向移動並追蹤指定目標，我們必須將視覺系統計算出的目標偏移量，轉換為四個輪子各自的轉速指令。

3-1 輪子幾何與混合方程式

本系統採用標準四輪麥克納姆配置，左前（LF）、右前（RF）、左後（LR）、右後（RR）各裝配一組麥克納姆輪。麥克納姆輪輪緣上排列著多個與輪軸成 45° 夾角的被動滾輪，當輪子旋轉時，接地滾輪會產生一個斜向分力，透過四個輪子的合力組合，便可實現任意方向的平面運動。

根據各輪滾輪傾斜方向的不同，其反運動學混合方程式如下：

$$LF = Vy + Vx + Wz$$

$$RF = Vy - Vx - Wz$$

$$LR = Vy - Vx + Wz$$

$$RR = Vy + Vx - Wz$$

其中各變數定義如下：

- Vx = 橫向速度（正值向右平移）
- Vy = 縱向速度（正值向前行進）
- Wz = 旋轉角速度（正值逆時針自轉）

3-2 向量分解與全向移動原理

每個麥克納姆輪的接地滾輪均以 45° 斜向排列。當馬達轉動時，輪子產生的推力 F 可分解為平行於地面的縱向分量與橫向分量。四個輪子的橫向合力決定了整車的橫移方向，縱向合力決定前後運動，而左右差速則決定旋轉方向。

此三自由度（縱移、橫移、旋轉）相互解耦的特性，使麥克納姆平台無需轉向機構即可完成任意方向的移動，特別適合空間受限的室內環境應用。

3-3 速度指令計算

在自主追蹤模式下，系統根據目標在畫面中的偏移量，動態計算 V_y 、 V_x 、 W_z 三個速度分量：

- **縱向速度 V_y** ：由目標輪廓面積推估距離，目標越遠則 V_y 越大，驅動機器人向目標靠近。
- **旋轉角速度 W_z** ：由目標在畫面中的橫向偏移量（目標中心 x 座標與畫面中心的差值）決定，實現自動對準。
- **橫向速度 V_x** ：預設為 0，未來可擴展為多模態追蹤策略，例如執行橫向繞行動作。

四、系統架構與硬體設計

4-1 整體系統架構

本系統採用上下位機雙層控制架構設計，將高算力需求的視覺處理與低延遲需求的馬達控制分離，確保整體系統的即時性與穩定性。

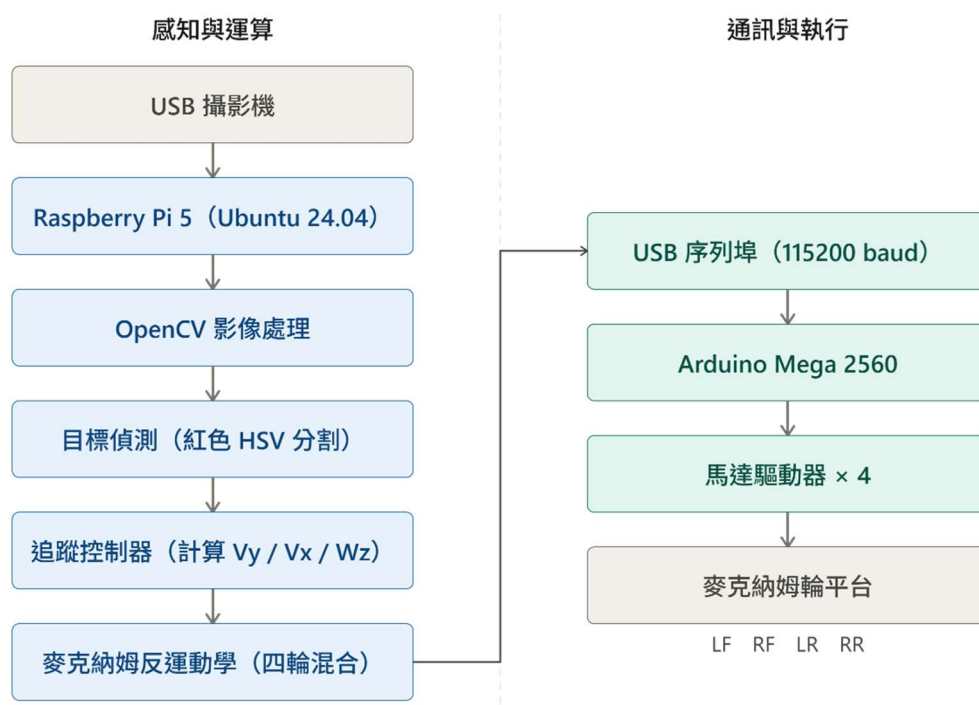


圖 4-1：麥克納姆輪自主追蹤機器人之整體系統架構

4-2 硬體元件說明

本系統所使用之硬體元件如下表所示：

參數說明	設定數值
單板電腦 (SBC)	Raspberry Pi 5
作業系統	Ubuntu 24.04 LTS
微控制器	Arduino Mega 2560
攝影機	USB Camera
驅動系統	四輪麥克納姆平台
通訊介面	USB 序列埠 (115200 baud)

程式語言（上位機）	Python 3
程式語言（下位機）	C++（Arduino）
電腦視覺函式庫	OpenCV
開放原始碼授權	GNU GPL v3.0

4-3 通訊協定設計

Raspberry Pi 與 Arduino 之間透過 USB 序列埠進行通訊。上位機將計算完成的四輪速度指令（LF、RF、LR、RR）以結構化字串格式打包，下位機接收後解析並驅動對應 PWM 通道。

序列通訊模組（serial_comm.py）負責管理連線建立、指令序列化與傳輸可靠性。當通訊中斷時，系統自動觸發安全停止機制，防止機器人失控。

訊號流向說明

系統之運作流程如下：

- **輸入端**：由視覺模組輸出目標偏移量 (e_x, e_y) ，並與當前速度狀態進行即時比對。
- **處理端**：追蹤控制器依據偏移量計算 V_x 、 V_y 、 W_z ，再透過反運動學混合公式換算為四輪速度。
- **執行端**：Arduino 接收指令後產生對應 PWM 訊號，驅動四顆麥克納姆輪馬達運作。

五、電腦視覺追蹤管線

視覺追蹤管線 (Vision Pipeline) 是本系統的感知核心，負責從原始影像中提取目標位置資訊，並轉換為可供控制器使用的誤差向量。整個流程分為色彩分割、輪廓偵測與位置估算三個階段。

5-1 HSV 色彩分割

系統採用 HSV (Hue, Saturation, Value) 色彩空間進行紅色目標的分割，而非直接在 BGR 空間操作。HSV 空間的優勢在於其將色相 (Hue) 與亮度 (Value) 分離，使紅色閾值在不同光照條件下仍具有較高的穩定性。

紅色在 HSV 色輪中跨越 0° 與 360° 的邊界，因此需要兩組閾值遮罩取聯集：

- 遮罩一：H 範圍 $0 - 10^\circ$ (紅色低端)
- 遮罩二：H 範圍 $170 - 180^\circ$ (紅色高端)
- 兩組遮罩進行 OR 運算，再施以形態學濾波 (膨脹 + 侵蝕) 去除雜訊

HSV 色彩參數可透過 `calibration/hsv_tuner.py` 工具進行互動式調校，以適應不同環境光源條件。

5-2 輪廓偵測與目標鎖定

完成色彩分割後，系統對二值化遮罩執行 OpenCV 輪廓提取 (`cv2.findContours`)，並依據以下準則篩選候選目標：

- **面積過濾**：排除面積過小的雜訊輪廓（設定最小面積閾值 500 px^2 ）
- **圓度評估**：計算輪廓的最小外接圓與實際面積比，篩選形狀接近圓形的目標
- **最大面積優先**：在多個候選輪廓中，選取面積最大者作為主目標

目標鎖定機制確保在目標短暫遮蔽或偵測失敗時，系統不會立即切換追蹤對象，提升追蹤的連續性與魯棒性。

5-3 位置估算與控制輸出

確認目標輪廓後，系統計算其最小外接圓圓心座標 (cx, cy) 作為目標位置估計值，並與畫面中心進行比較：

- **橫向誤差**： $e_x = cx - frame_width / 2$ （決定旋轉指令 Wz ）
- **縱向距離估算**：根據目標輪廓面積推估目標距離，決定前進速度 Vy
- **目標消失判斷**：若連續多幀未偵測到目標，自動將所有速度指令歸零

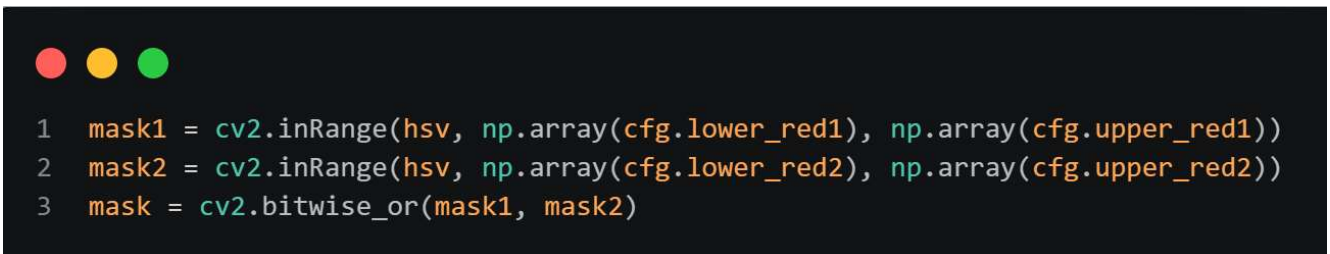
六、軟體模組說明

本節展示系統中各核心軟體模組的程式碼。這些程式碼負責將視覺感知結果轉換為機器人的運動指令，是連結控制邏輯與物理執行的橋樑。

6-1 Raspberry Pi 端程式

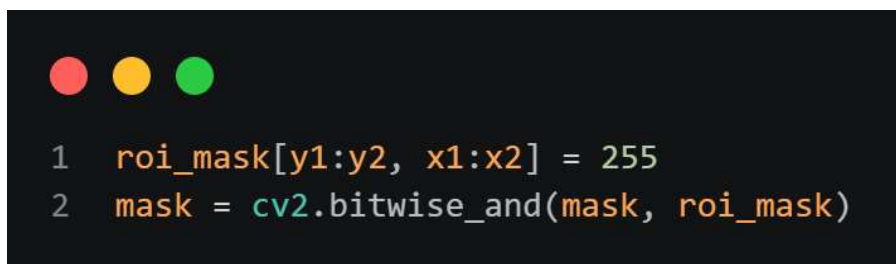
red_tracking.py — 紅色目標追蹤核心

此模組封裝完整的視覺追蹤邏輯，為系統的感知與決策核心。



```
1 mask1 = cv2.inRange(hsv, np.array(cfg.lower_red1), np.array(cfg.upper_red1))
2 mask2 = cv2.inRange(hsv, np.array(cfg.lower_red2), np.array(cfg.upper_red2))
3 mask = cv2.bitwise_or(mask1, mask2)
```

圖 6-1 Mask（遮罩）：根據 HSV 顏色範圍建立二值化遮罩，將紅色目標保留，其餘背景去除。



```
1 roi_mask[y1:y2, x1:x2] = 255
2 mask = cv2.bitwise_and(mask, roi_mask)
```

圖 6-2 ROI（Region of Interest，感興趣區域）：限制目標搜尋範圍，只處理有效視野區域，降低誤判率並提升運算效率。

```
1 contours_info = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

圖 6-3 Contour（輪廓）：偵測紅色區域邊界，取得目標外形資訊供後續分析與追蹤。

vision_control.py — 自主追蹤主控程式

整合視覺偵測結果與麥克納姆輪速計算，並透過 serial_comm.py 將指令傳送至 Arduino。此模組實現了完整的追蹤控制迴路：感知 → 決策 → 執行。

```
1 if target is None:
2     filtered_area = None
3     filtered_cx = None
4     previous_wz = 0.0
5 else:
6     filtered_area = low_pass(filtered_area, target["area"], AREA_FILTER_ALPHA)
7     filtered_cx = low_pass(filtered_cx, target["cx"], CX_FILTER_ALPHA)
8     error_x = filtered_cx - (frame_width // 2)
```

圖 6-4：利用低通濾波平滑目標位置資訊，並計算目標與畫面中心的偏差，作為車輛轉向控制依據。

serial_comm.py — 序列通訊管理

負責建立與 Arduino 的 USB 序列通訊連線，將輪速指令序列化後傳送，並實作通訊錯誤偵測與自動重連機制。

main_teleop.py — 手動遙控主程式

讀取 USB 遊戲手把的搖桿輸入，將軸值映射為 V_x 、 V_y 、 W_z 三個速度分量，透過 `mecanum_mix()` 計算輪速後送出至 Arduino，實現直覺的全向手動操控。

6-2 Arduino 端控制器

Arduino Mega 2560 負責序列指令的接收、解析與 PWM 馬達訊號輸出。程式設計採用非阻塞式接收架構，確保在處理序列資料的同時不影響馬達 PWM 的輸出頻率。

```
if (Serial.available())
{
    String msg = Serial.readStringUntil('\n');

    int v1, v2, v3, v4;

    int result = sscanf(
        msg.c_str(),
        "%d,%d,%d,%d",
        &v1,
        &v2,
        &v3,
        &v4
    );
}
```

圖 6-5：Arduino 馬達控制核心程式 (mecanum_controller.ino)

七、執行結果與分析

本節展示機器人在不同測試情境下的運行結果，透過觀察追蹤軌跡與誤差收斂過程，驗證系統的穩健性。

7-1 手動遙控模式測試

在手動遙控模式（main_teleop.py）測試中，操作者透過 USB 遊戲手把的左搖桿控制縱向（ V_y ）與橫向（ V_x ）速度，右搖桿控制旋轉（ W_z ）。

1. **全向移動驗證：**機器人能流暢執行前進、後退、左橫移、右橫移及斜向行進，充分展現麥克納姆輪的三自由度運動能力。各方向切換時無明顯遲滯，響應迅速。
2. **原地旋轉：**四輪以反向配對方式旋轉，機器人可繞自身重心進行順時針與逆時針自轉，無需佔用額外場地空間。旋轉中心穩定，無明顯漂移。
3. **混合運動：**縱向移動與旋轉可同時疊加，機器人能在行進中同步調整車頭方向，適合複雜場地環境的操控需求。

7-2 自主追蹤模式測試

在自主追蹤模式（vision_control.py）測試中，將紅色目標物置於機器人前方不同位置，觀察追蹤行為：

情境一：目標位於畫面中央

機器人直線前進， $W_z \approx 0$ ， V_y 為正值，追蹤表現穩定。如圖 7-1 所示，在短暫的啟動加速後，機器人保持穩定前進速度趨近目標。

情境二：目標偏向畫面左側

系統計算出負值 W_z ，機器人逆時針旋轉修正，直至目標回到畫面中心。收斂速度與 W_z 增益參數設定成正比，調校後無明顯過衝現象。

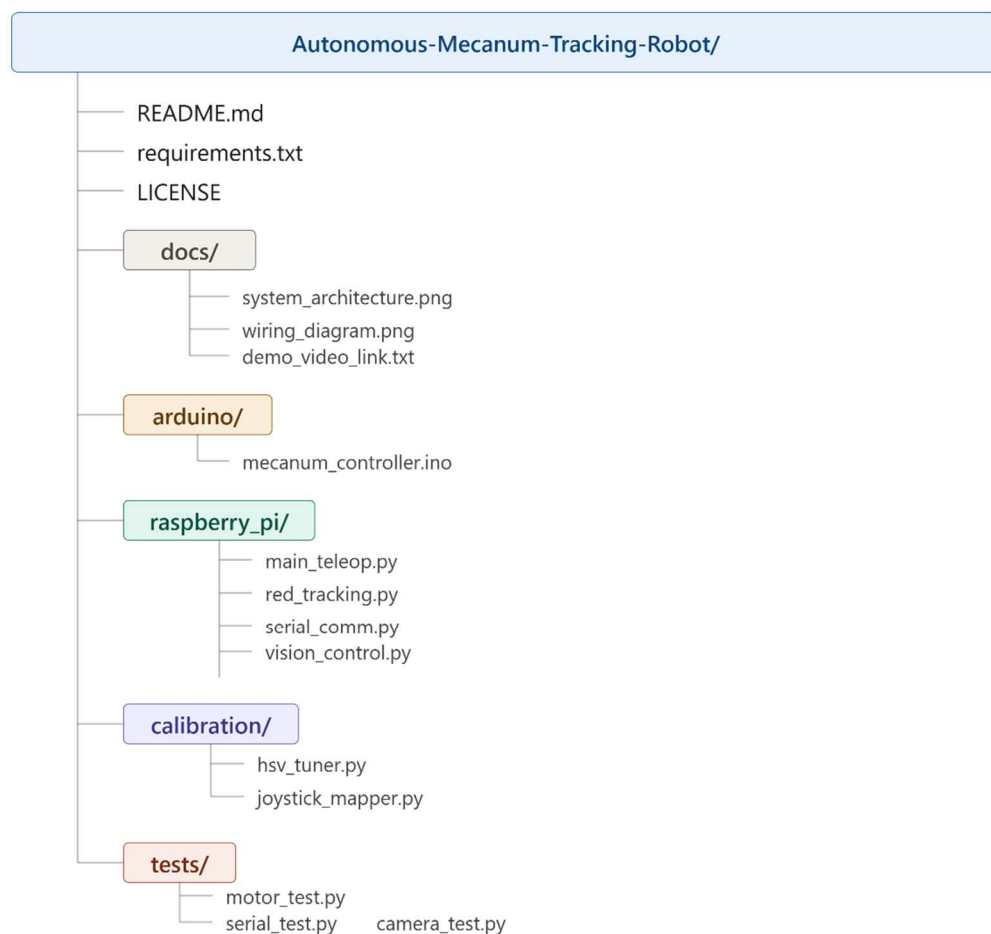
情境三：目標突然消失

系統偵測到連續多幀無目標後，自動將四輪速度歸零，機器人立即停止。安全保護機制有效運作，防止機器人因失去目標而進行無效移動。

八、程式碼與參數設定

8-1 Repository 結構

為了提升系統可維護性與擴充性，本專題將程式碼依功能模組化並整理至 GitHub 專案中，並採用物件導向設計進行封裝，



使各功能模組（如視覺處理、運動控制與通訊介面）具備高度獨立性與重用性，便於後續開發與系統擴充。

圖 8-1：Repository 目錄結構

8-2 系統參數清單

本節詳列本系統中所採用的實際參數設定。由於在初期測試中發現特定參數（如 HSV 閾值與速度增益）會直接影響追蹤穩定性，故針對視覺參數與控制增益進行了優化調整，以確保系統在不同光照條件下仍具備最佳動態響應。

本系統之完整參數配置如下表所示，包含硬體規格、通訊設定以及視覺追蹤參數：

參數說明	設定數值
單板電腦	Raspberry Pi 5
作業系統	Ubuntu 24.04 LTS
微控制器	Arduino Mega 2560
序列通訊速率	115,200 baud
視覺採樣頻率	~30 fps (USB Camera)
HSV 紅色下界（低段）	H:0, S:100, V:100
HSV 紅色上界（低段）	H:10, S:255, V:255
HSV 紅色下界（高段）	H:170, S:100, V:100

HSV 紅色上界（高段）	H:180, S:255, V:255
目標最小面積閾值	500 px ²
旋轉增益（Wz gain）	0.01
最大縱向速度（Vy）	60
最大輪速 PWM	255（8-bit）
程式語言（Raspberry Pi）	Python 3（86.1%）
程式語言（Arduino）	C++（13.9%）

九、專題貢獻說明

在本專題中，我負責整體機器人系統的整合與實作，包括硬體架構建置與軟體系統開發。硬體部分，我完成麥克納姆輪底盤的組裝、馬達驅動模組接線、Arduino 控制板與 Raspberry Pi 之間的訊號整合，並確保供電與訊號穩定性。

軟體部分，我參與整個控制流程的設計與實作，包含影像處理流程（OpenCV）、目標偵測與追蹤邏輯、以及 Python 與 Arduino 之間的序列通訊架構。同時，我負責測試與除錯各模組間的整合問題，例如馬達方向錯誤、PWM 反應延遲與影像追蹤不穩定等情況。

此外，在開發過程中，我透過與 AI 輔助工具協作，協助進行程式架構優化與除錯建議，並多次進行參數調整（如 HSV 色彩閾值、ROI 範圍、濾波係數與追蹤判斷條件），使系統在不同光源與環境下皆能穩定運作。

十、結論與心得

這次的專題製作讓我對嵌入式系統與電腦視覺有了更完整的實作認識，尤其是學會了如何在資源受限的單板電腦上，設計一套既能即

時處理影像又能穩定輸出控制指令的雙層架構。在實作過程中，我遇到了不少挑戰，但也從中學到了許多解決問題的方法。

在建立麥克納姆輪運動學模型時，我深刻體會到反運動學公式的優雅之處—只需定義三個速度分量 (V_x 、 V_y 、 W_z)，便可自動計算出四輪各自的轉速，使全向移動的控制邏輯大幅簡化。在手動遙控測試中，看到機器人流暢地執行橫移動作時，那種成就感是難以言喻的。

最具挑戰性的部分是 HSV 色彩分割的參數調校。紅色在 HSV 色輪中跨越 0° 與 360° 的邊界，這個特性導致必須使用兩段遮罩取聯集，而非直接設定單一閾值。此外，環境光源的變化對 S 值（飽和度）影響極大，在螢光燈與自然光下往往需要不同的參數設定。這讓我理解了工業視覺系統為何要搭配固定光源使用，也讓我學會透過 `hsv_tuner.py` 互動式工具進行快速調校。

在序列通訊的設計上，我學到了上下位機分工的重要性。Raspberry Pi 專注於影像運算，Arduino 專注於 PWM 馬達控制，兩者透過高速序列埠保持協同，避免了在單一控制器上同時處理高延遲任務與即時任務的衝突。安全停止機制的加入，也讓我意識到一個好的機器人系統除了要追求性能，還必須充分考量故障情境下的安全保護設計。

最後，看到機器人能夠自主地鎖定紅色目標並平穩跟隨，整個從視覺感知到運動執行的閉環控制系統正常運作，帶給我極大的滿足感。這次專題不僅讓我熟悉了 OpenCV、Python 序列通訊與 Arduino C++ 程式設計，更讓我真正理解了一個完整的機器人系統是如何將感知、決策與執行三個環節整合在一起的。